

RAG Cheat Sheet (1-page quick reference)

Pipeline: Index (load chunk embed store) · Retrieve (embed query search rerank) · Generate (grounded prompt answer + citations).

Defaults that are "right until proven otherwise" (2026)

- **Chunking:** recursive, ~512 tokens, 10–20% overlap. Sweep 256/512/1024, pick by RAGAS.
- **Embeddings:** text-embedding-3-small (1536-d). Same model for index & query.
- **Retrieval:** hybrid (vector + BM25) **RRF cross-encoder rerank**. Retrieve ~20, rerank to ~4.
- **Generation:** temperature 0, grounded prompt, citations, explicit refusal, relevance gate.
- **Vector DB:** Chroma (proto) · pgvector (on Postgres) · Qdrant/Weaviate (dedicated) · Pinecone (managed) · Milvus (billions).
- **Index type:** HNSW (default) · IVF/PQ (memory at scale).

The numbers to quote

- 80% of RAG failures originate in ingestion/chunking.
- Chunking can swing accuracy ~80%+; embeddings ~20%.
- Hybrid adds ~1–9% recall over pure vector.
- Latency target: **TTFT p90 < 2s**; track p50/p90/p99.
- Golden set: 30–50+ reviewed Q&A pairs.

Four eval metrics (RAGAS)

Metric	Catches	Low fix
Faithfulness	hallucination	grounding prompt + gate
Answer relevancy	off-topic	prompt/format
Context precision	junk retrieved	rerank/hybrid/filters
Context recall	missed info	chunking/top-k

Precision+recall = retrieval health · Faithfulness+relevancy = generation health.

Failure fix (advanced)

- Query/doc asymmetry **HyDE** · phrasing gaps **multi-query** · multi-part **decomposition**
- Small-vs-context tension **parent/small-to-big** · noisy chunks **compression**
- Context-dependent chunks **contextual retrieval** · bad retrieval **CRAG** · cost **adaptive routing**

Retrieval cheat facts

- Dense = meaning/paraphrase; Sparse/BM25 = exact terms/codes/names. Combine = hybrid.
- RRF fuses by **rank** ($1/(k+\text{rank})$), ignoring score scales.
- Bi-encoder = retrieve wide/cheap; Cross-encoder = rerank narrow/accurate.
- "Lost in the middle" rerank to few, most-relevant first.

Agentic

- **Routing (Adaptive RAG)**: cheapest sufficient path per query — top cost control.
- **Loop = ReAct**: thought action observation. Always: **max-step budget + fallback**.
- Use an agent only when single-shot retrieval provably can't answer.

Graph RAG

- Triples (subject, relation, object) graph traverse. Wins on **multi-hop** & **global** Qs.
- Costly (extraction, entity resolution, freshness). Usually hybridized with vectors.

Production must-haves (Doc 11)

Separate **indexing vs query** pipelines · **semantic cache** · **guardrails** (injection/PII, in & out) · **tracing** per request · **metrics** (latency %iles, cache hit, refusal rate, cost/query) · **access control** via metadata filters · **online eval** + **CI eval** · idempotent **upserts** + **deletes** (right-to-be-forgotten).

Security one-liners

- Per-tenant **metadata pre-filter** = no cross-tenant leak.
- Treat retrieved text as **untrusted data, not instructions** = injection defense.
- Redact **PII** before it reaches the LLM / in outputs.

Interview reflexes

1. Clarify requirements first.
2. Both pipelines.
3. A metric per component.
4. A trade-off + a number per decision.
5. Say how you'd evaluate & monitor it.